SC3050 Advanced Computer Architecture

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Review: Single-Cycle, Pipelining and Hazards

From one long clock to an assembly line: why we pipeline, and what breaks when we do.

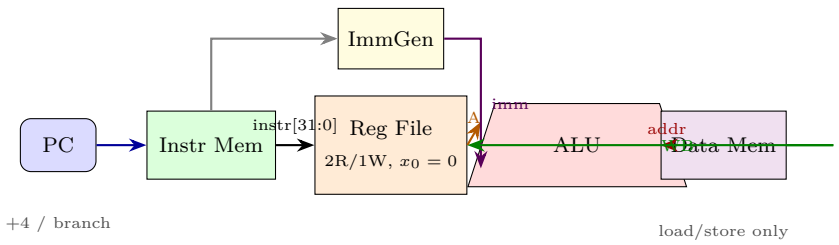
1.1 SC1006 Recap and Roadmap

SC1006 built the RV32I machine from the ground up: ISA formats, single-cycle datapath, hardwired control, and the classic 5-stage pipeline. SC3050 starts where overlap stops being free. This chapter distils the single-cycle baseline, the ideal pipeline, and the three hazard families that force real hardware to stall, forward, or predict.

We assume RV32I, 32 registers ($x_0 \equiv 0$), and byte-addressed memory. All timing numbers are illustrative; only the structure matters.

1.2 Single-Cycle Baseline

In a single cycle every instruction traverses the same unified datapath: PC → instruction memory → register file → ALU → data memory → register file write-back. Control is purely combinational: opcode, funct3 and funct7 select `RegWrite`, `ALUSrc`, `MemRead`, `MemWrite`, `MemToReg`, `Branch` and `Jump`.



Definition 1.1 (Critical path)**.** $T_{\text{clk}} \geq t_{\text{IM}} + t_{\text{RF}} + t_{\text{ALU}} + t_{\text{DM}} + t_{\text{setup}}$ for single-cycle. `LW` traverses every block and therefore sets the period; `ADD` and `BEQ` finish early but still wait.

Throughput is $1/T_{\text{clk}}$ instructions per second; CPI = 1 by construction but T_{clk} is long and utilisation is poor: the ALU idles during IF, memory idles during EX for R-type, and so on.

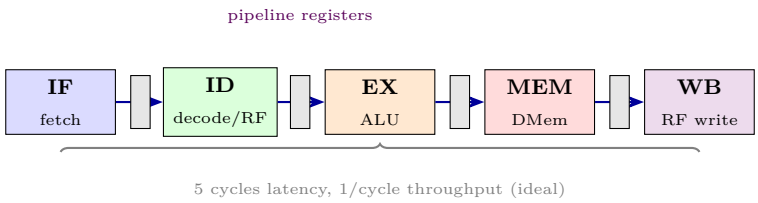
The immediate generator and PC logic illustrate the combinational depth. RV32I has six immediate formats (I, S, B, U, J, and the shamt variant); ImmGen is a network of muxes and sign-extenders selected by opcode. The PC source mux chooses among PC+4, branch target (PC + sign-extended B-imm) when `Branch`$\wedge$`Zero`, and jump target for `JAL`/`JALR`. In single-cycle all of this settles before the rising edge that writes the register file. Any setup or hold violation would corrupt state, so T_{clk} includes Worst-case margins.

Stage in single-cycle	Active blocks	What happens in that fraction of T_{clk}
Fetch	PC, I-Mem, adder	PC drives I-Mem, word fetched, PC+4 computed
Decode	ImmGen, RegFile read	opcode decoded, rs1/rs2 read, imm formed
Execute	ALU, comparator	ALU computes, branch condition evaluated
Memory	D-Mem	load/store touches memory; others idle
Write-back	RegFile write	result selected by `MemToReg` and written if `RegWrite`

Splitting this monolith is what creates the pipeline. Each stage becomes a clock domain separated by a register, and T_{clk} shrinks to the worst stage rather than the sum.

1.3 The Ideal 5-Stage Pipeline

Insert registers IF/ID, ID/EX, EX/MEM, MEM/WB between the five logical stages. Each holds the instruction word, PC+4, register operands, immediates, and control bits for the next stage.



With k instructions and $n = 5$ stages each t plus register overhead t_r, pipelined time $\approx (n + k - 1)(t + t_r)$ versus knt sequential; speedup $\to n$ as $k \to \infty$ but unequal stage delays and hazards reduce it.

Instr	1	2	3	4	5	6	7
`LW x5,0(x6)`	IF	ID	EX	MEM	WB		
`ADD x7,x5,x8`		IF	ID	EX	MEM	WB	
`SW x9,4(x7)`			IF	ID	EX	MEM	WB
`BEQ x7,x0,T`				IF	ID	EX	MEM

Ideal CPI = 1; real CPI = $1 + s$ where s is stall cycles per instruction. Forwarding and prediction minimise s.

1.4 Hazards

Definition 1.2 (Hazard classes)**.** *Structural*: resource conflict; *data* (RAW, WAR, WAW): operand not ready or name conflict; *control*: next PC unknown. In-order 5-stage sees only structural and RAW; WAR/WAW appear with OOO issue.

1.4.1 Structural Hazards

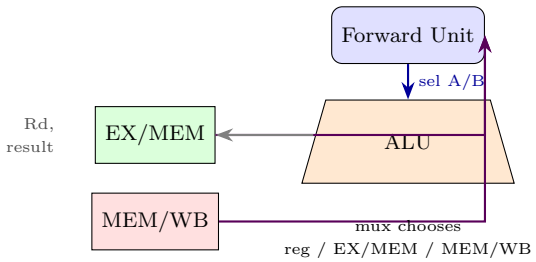
Two instructions need the same resource in the same cycle. The classic is a single-ported memory serving both IF and MEM. Fix: split I/D memories (Harvard at L1) or stall. Modern cores also duplicate adders and ports to avoid structural conflicts at issue width.

1.4.2 Data Hazards and Forwarding

RAW is the common case: j reads a register before i writes it.

Listing 1.1: RAW hazards: forwarding vs. load-use stall

```
ADD  x5, x6, x7       # x5 produced in EX/MEM
SUB  x8, x5, x9       # RAW on x5 -> forward from EX/MEM, no stall
LW   x5, 0(x1)        # x5 from MEM
ADD  x6, x5, x2       # load-use: cannot forward in time -> 1 bubble
BEQ  x5, x0, target # branch reads x5 -> forward or stall if load
```



The hazard unit compares $\mathrm{Rd}_{\mathrm{EX}}$ and $\mathrm{Rd}_{\mathrm{MEM}}$ against $\mathrm{Rs1}_{\mathrm{ID}}/\mathrm{Rs2}_{\mathrm{ID}}$ and steers muxes. Load-use ($\mathrm{Rd}_{\mathrm{EX}}$ is load destination and $\mathrm{Rs}_{\mathrm{ID}}$ matches) cannot be fixed by forwarding: data arrives from MEM after EX has passed, so the pipeline stalls IF/ID for one cycle and injects a bubble (`nop`) into ID/EX.

A compiler can also hide latency by scheduling independent instructions between a producer and consumer.

1.4.3 Control Hazards

Branches resolve late (EX or MEM in the simplest pipeline). Instructions fetched after the branch may be on the wrong path. Mitigations: move compare to ID (1-cycle penalty), flush on mispredict, or predict.

Listing 1.2: Compiler scheduling hides a load-use hazard

```
# before scheduling (1 stall needed)
LW   x5, 0(x10)
ADD  x6, x5, x11      # stall
SUB  x7, x8, x9       # independent
# after scheduling (no stall)
LW   x5, 0(x10)
SUB  x7, x8, x9       # independent work fills the slot
ADD  x6, x5, x11      # x5 now ready via forwarding from MEM/WB
```

Flushing zeroes control bits in IF/ID and ID/EX. Prediction (Ch. 3) turns the flush from always-taken to rarely-taken.

1.5 Performance Model with Hazards

Let $n = 5$, clock $T_c = \max_i t_i + t_r$, ideal CPI $= 1$, stall rate $s = s_{\text{data}} + s_{\text{ctrl}} + s_{\text{struct}}$. Then CPI $= 1 + s$, time $= (n + k - 1 + sk)T_c$, and speedup over single-cycle is limited by $\max_i t_i$ imbalance and by s. For a mix with 20% loads and 15% branches, even a perfect predictor can leave $s \approx 0.1$ from load-use alone.

1.6 Stall and Bubble Mechanics

When the hazard unit detects a load-use, it asserts three controls in the same cycle: keep PC unchanged (no fetch), keep IF/ID latched (replay the consumer), and inject a bubble (all-zero control word) into ID/EX. The bubble behaves like `ADDI x0,x0,0` and propagates harmlessly. One bubble suffices because after one stall the load's data is in MEM/WB and can be forwarded.

Cycle	Instr	Hazard?	Forward	Stall	Action
1	`LW x5,0(x1)`	—	—	—	—
2	`ADD x6,x5,x2`	RAW load-use	no	yes	bubble in ID/EX
3	`ADD x6,x5,x2`	—	MEM/WB→EX	no	resume

WAR and WAW do not occur in a classic in-order pipeline because writes happen in order in WB; they appear once we allow out-of-order issue (Ch. 2) and require renaming.

1.7 Forwarding Conditions in Detail

The forward unit implements:

$$\text{ForwardA} = 01 \iff \text{RegWrite}_{\text{EX/MEM}} \wedge \text{Rd}_{\text{EX/MEM}} \neq 0 \wedge \text{Rd}_{\text{EX/MEM}} = \text{Rs1}_{\text{ID/EX}}$$

$$\text{ForwardA} = 10 \iff \text{RegWrite}_{\text{MEM/WB}} \wedge \text{Rd}_{\text{MEM/WB}} \neq 0 \wedge \text{Rd}_{\text{MEM/WB}} = \text{Rs1}_{\text{ID/EX}} \wedge \text{not case 01}$$

ForwardB analogous for Rs2.

Priority favours EX/MEM over MEM/WB because it holds the newer result. Loads forward from MEM/WB; ALU results forward from EX/MEM one cycle earlier, which is why `ADD; SUB` needs no stall but `LW; ADD` does. The zero register is excluded to avoid false matches. Extra muxing adds perhaps 0.1–0.2 ns to T_c but saves many stall cycles.

1.8 Quantitative Comparison

Example: $t_{\text{IM}} = 2.0$, $t_{\text{RF}} = 1.2$, $t_{\text{ALU}} = 1.8$, $t_{\text{DM}} = 2.4$ ns, single-cycle $T_{\text{clk}} \approx 8$ ns. Pipelined $T_c = \max(t_i) + t_r \approx 2.6$ ns, ideal speedup $\approx 3.1\times$ before stalls. With $s = 0.15$ CPI becomes 1.15, effective time per instruction $1.15 \times 2.6 \approx 3.0$ ns versus 8 ns still $2.7\times$ faster despite hazards. Unequal stages waste slack: IF and EX idle part of T_c. Balancing stages or using deeper pipelines (7–14 stages) helps until latch overhead and hazards dominate, which is why modern cores add superscalar width instead of extreme depth.

1.9 Control Signals Through the Pipeline

Unlike single-cycle, control is not monolithic. Signals are generated in ID from opcode/funct and latched forward:

Stage	Control produced/held	Example signals
ID	generate all	`ALUOp, ALUSrc, RegWrite, MemRead/Write, Branch`
EX	ALU control	`ALUCtrl`, `ALUSrc` mux, branch compare
MEM	memory enables	`MemRead, MemWrite, Branch+Zero` $\rightarrow$ `PCSrc`
WB	write-back	`RegWrite, MemToReg`

If an instruction is flushed, its pipeline register control bits are zeroed so it writes nothing and changes no state. Exceptions are handled similarly: an illegal opcode detected in ID causes all younger instructions to be flushed and the trap handler address to be injected into PC. Precise exceptions require that instructions before the fault retire and those after are squashed, which is straightforward here because writes are in order.

1.10 Why Pipelining Alone Plateaus

Deeper pipelines (10–20 stages) reduce T_c but face three walls: (1) latch overhead t_r is not scalable, (2) branch penalty grows with depth because more speculative instructions are in flight, and (3) load-use penalty may grow if MEM is far from EX. The historical answer moved from frequency scaling to width scaling: issue multiple instructions per cycle (superscalar), allow out-of-order completion (Tomasulo), and speculate past branches. Each builds on the forwarding and flushing primitives reviewed here. The next chapters add those mechanisms one by one.

1.11 Key Takeaways

Single-cycle sets T_c by the slowest instruction. Pipelining cuts T_c toward $\max_i t_i$ and overlaps instructions but exposes hazards. Structural hazards are solved by replication; data hazards by forwarding, stalls, and scheduling; control hazards by early resolution and prediction. The rest of SC3050 removes these ceilings: superscalar and OOO attack data hazards, branch prediction attacks control hazards, and coherence attacks the memory wall.

Remark 1.1 (Exam checklist). Given any short RV32I fragment, you should be able to label structural / RAW / control hazards, state the forwarding path or stall count, and compute CPI $= 1 + s$. Practice by annotating every producer–consumer pair with its distance in cycles and whether a bubble is needed.

Bridge: the in-order pipeline saturates at CPI $\approx$ 1.1–1.3 for many workloads. To break through we must fetch and execute multiple instructions per cycle and allow them to overtake each other — the subject of Chapter 2.

1.12 Exercises

Exercise 1.1. For a 5-stage pipeline with $t_{\mathrm{IF}} = 2.0$, $t_{\mathrm{ID}} = 1.2$, $t_{\mathrm{EX}} = 1.8$, $t_{\mathrm{MEM}} = 2.4$, $t_{\mathrm{WB}} = 1.0\,\mathrm{ns}$ and $t_r = 0.2\,\mathrm{ns}$, what is T_c? What is latency for one instruction and throughput in ideal steady state? Which stage limits T_c?

Exercise 1.2. Sequence: `LW x5,0(x1)`; `ADD x6,x5,x2`; `SUB x7,x6,x3`. Identify every RAW hazard, state whether forwarding resolves it or a stall/bubble is required, and draw the 7-cycle timeline with bubbles.

Exercise 1.3. With a single memory for IF and MEM, construct a cycle where a structural hazard occurs. Show two fixes (stall vs. split cache) and quantify the CPI impact for a workload with 30% loads/stores if the stall fix adds one bubble per MEM-stage instruction.

Exercise 1.4. Explain why `x0` needs special treatment in the hazard unit. Give an instruction pair where $\mathrm{Rd}_{\mathrm{EX}} = x_0$ matches $\mathrm{Rs1}_{\mathrm{ID}}$ but must not trigger forwarding or a stall.

Exercise 1.5. Branch penalty is 2 cycles, 18% of instructions are branches, predictor accuracy 75%. Compute average stall per instruction due to control hazards. How much does accuracy 95% save? Relate to CPI $= 1 + s$.

Chapter 2

Superscalar, Out-of-Order and Tomasulo

One per cycle is not the ceiling: fetch more, execute when ready, and commit in order.

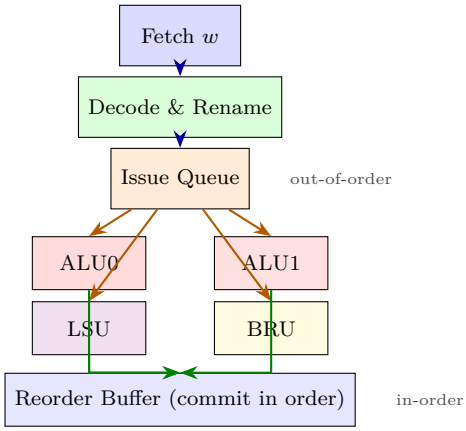
2.1 Why Go Wider and Out-of-Order?

An in-order 5-stage pipeline tops out at IPC = 1 (instructions per cycle). Real programs have parallel work, but hazards force stalls. Two ideas raise throughput: (1) **superscalar** — fetch, decode and execute w instructions per cycle ($w = 2$–8 in 2026 cores), and (2) **out-of-order execution** — let a ready instruction overtake a stalled one, then restore program order at commit.

Wider without OOO wastes slots: a cache miss or RAW stalls the whole bundle. OOO without width leaves execution units idle. Together they exploit instruction-level parallelism (ILP) limited only by true data dependences (RAW); WAR and WAW are name dependences removable by renaming.

2.2 Superscalar Organisation

A w-way superscalar duplicates decode logic and has multiple functional units (e.g., 2 ALU, 1 load/store, 1 branch, 1 FP). In each cycle up to w instructions are fetched (needs wide I-cache and branch prediction), decoded, checked for dependences, and issued if operands and units are available.



Key hazards at width: data dependences within the bundle (need bypass within the group), structural misses (not enough ALUs), and control (one taken branch per group limits fetch). Issue logic must wake and select among ready instructions each cycle; selection priority affects performance but not correctness.

Typical server mix at width 4: two integer ALUs, one FP/vector, one branch, two load/store ports (one for address generation). Compiler and predictor must supply enough independent instructions; otherwise issue slots waste as `nop`. Fetch alignment matters: a taken branch in the middle of a 16-byte fetch block discards the tail, and an unaligned w-wide fetch may cross two I-cache lines.

Resource	Count at $w = 4$	Request per cycle (typical)	Stall if exceeded
Integer ALU	2	1.2	structural
Load/store ports	2 (1 AGU)	0.8	address queue backs up
Branch unit	1	0.2	fetch caps at one taken

2.3 Register Renaming Removes False Dependences

WAR and WAW arise only because ISA registers are reused. Renaming maps each write to a fresh physical register; later readers get the latest mapping. Example:

Listing 2.1: Renaming eliminates WAR/WAW

```
# ISA registers (before)          # after renaming (physical p)
ADD  x1, x2, x3      # x1=W      -> ADD  p10, p2, p3
SUB  x1, x4, x5      # WAW on x1 -> SUB  p11, p4, p5  (no conflict with p10)
LW   x6, 0(x1)       # WAR on x1 -> LW   p12, 0(p11)  (uses new p11)
ADD  x7, x1, x6                   -> ADD  p13, p11, p12
```

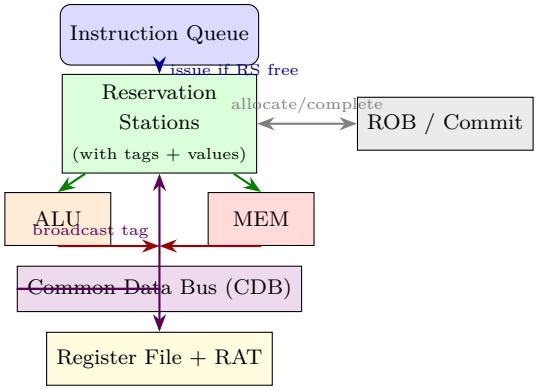
A Register Alias Table (RAT) holds the current ISA→physical mapping; a free list supplies new physical registers on each rename. Reclamation happens at commit. With enough physical registers, only RAW remains.

Definition 2.1 (Precise state)**.** Precise exceptions require that when an instruction traps, all earlier instructions have committed and no later instruction has changed architectural state. OOO hardware achieves this by committing in order via the ROB, discarding speculative results on a trap.

Without renaming, even a tiny loop `for(i) a[i]=b[i]+c` would serialize on reuse of temporaries. Renaming lets successive iterations overlap in the window, which is why unrolling is less critical on OOO than on in-order VLIW. The number of physical registers (e.g., 128 integers + 128 FP in a modern core) directly bounds how far ahead the window can see.

2.4 Tomasulo's Algorithm

Robert Tomasulo (IBM 360/91, 1967) invented the first OOO scheme that also handles renaming implicitly via reservation stations (RS) and a Common Data Bus (CDB).



Each RS entry holds: busy, opcode, V_j/V_k (values) or Q_j/Q_k (tags of producers), destination tag, and address/immediate. Issue stalls if no RS is free. An instruction waits in RS until both operands are ready (either present at issue or arriving via CDB). It then executes, broadcasts its result and tag on the CDB, and every waiting RS and the RAT snoops the CDB to capture the value and clear its tag. Later, the ROB retires results in program order.

Listing 2.2: Tomasulo wake-up and select (simplified)

```
# Each cycle:
for rs in reservation_stations:
    if rs.Qj == completed_tag: rs.Vj, rs.Qj = CDB.value, None
    if rs.Qk == completed_tag: rs.Vk, rs.Qk = CDB.value, None
# select ready
ready = [rs for rs in RS if rs.busy and rs.Qj is None and rs.Qk is None]
if ready: execute(choose_oldest(ready))  # priority by age
# CDB broadcast (one per cycle in classic design)
broadcast(tag, value)  # all RS + RAT update
```

Modern cores generalise Tomasulo: separate issue queues per functional unit, multiple CDBs/bypass networks, and a physical register file with the RAT. The ROB (or retirement register file) integrates speculation support.

2.5 Reservation Station Fields and Lifecycle

Each RS entry is a tiny dataflow node:

Field	Meaning
`Busy`	entry in use
`Op`	operation (ADD, MUL, LD, ...)
`Vj, Vk`	operand values when ready
`Qj, Qk`	tags of producers when not ready (0 ⇒ ready)
`Dest`	tag for this result (ROB index or physical register)
`A`	immediate / address offset

Lifecycle: **Issue**: allocate RS+ROB, read RAT for sources (if RAT maps to a pending tag, copy tag to Q; otherwise copy value to V), write destination tag into RAT. **Wait**: snoop CDB; when Q matches broadcast

tag, latch value and clear Q. **Execute**: when Qj=Qk=0 and FU free, launch. **Write-back**: broadcast Dest tag + result on CDB; RS marked free but ROB entry holds result until commit. **Commit**: in-order, copy ROB result to architectural state (or mark physical register as committed) and free the old physical mapping.

The ROB decouples OOO execution from in-order commit and provides buffering for precise exception recovery: on a mispredict or exception, ROB entries after the fault are flushed, RAT is rolled back via a checkpoint, and RS entries are cleared.

2.6 Superscalar Hazards in a Bundle

Within one w-wide fetch group, RAW can be intra-bundle. Example $w = 2$:

Slot 0	Slot 1	Issue?
`ADD x1,x2,x3`	`ADD x4,x1,x5`	RAW intra-bundle: second needs forwarding from first within same cycle or must issue next cycle
`LW x1,0(x2)`	`ADD x3,x1,x4`	load-use even stricter: 1-cycle bubble even across bundles
`ADD x1,x2,x3`	`MUL x6,x7,x8`	no dependence: dual issue OK if FU available

Complexity of dependence checking grows as $O(w^2)$; that is why w beyond 6–8 gives diminishing returns relative to power and bypass network cost. Compilers help by scheduling independent instructions together.

2.7 Memory Disambiguation and LSQ

Loads and stores must respect memory order. A Load-Store Queue (LSQ) tracks addresses: a load may bypass an earlier store only if addresses are known distinct; otherwise it waits. Store-to-load forwarding allows a load to take its value directly from an earlier store in the LSQ before the store reaches cache.

The LSQ is itself a reservation-station-like structure split into a load queue and store queue. Stores sit in the store queue until commit, then write to cache; loads can execute speculatively and may be replayed if a later-resolved store is found to alias. Memory dependence prediction (store sets) learns which loads tend to collide with which stores to avoid replays.

2.8 Performance and ILP Limits

Ideal ILP is limited by RAW chains. A classic estimate: IPC $\leq w$ but also ≤ 1/average dependence distance and $\leq$ FU count. Loops with cross-iteration dependences (`x[i]=x[i-1]+c`) expose little ILP; unrolling or vectorisation helps. Real cores sustain IPC $\approx$ 1.5–3.5 on integer code, 2–4 on FP, far below w because of branches and cache misses. Larger ROBs (224 entries in Apple M2, 512 in Intel Golden Cove) cover more miss latency and expose more ILP at the cost of power and wake-up selection complexity ($O(n^2)$ wake-up).

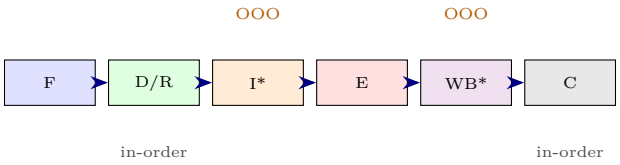
2.9 Scoreboard versus Tomasulo, and Modern Hybrids

The CDC 6600 *scoreboard* centralises hazard detection: issue stalls when WAR or WAW would occur, operands are read from the register file when available, and forwarding is explicit. Tomasulo distributes waiting into RS entries and uses tags to wait for data wherever it is produced, implicitly renaming registers so WAR/WAW never stall issue (only RAW and structural do). The price is associative wake-up and the CDB broadcast network. Modern cores use a hybrid: a physical register file with explicit RAT (like classic renaming) plus per-FU issue queues that behave like distributed RS. The key invariant remains: any instruction whose sources are ready may execute, regardless of program order, as long as commit is in order.

2.10 Putting It Together: OOO Pipeline Stages

Fetch → Decode → Rename → Dispatch (allocate RS+ROB) → Issue (out-of-order) → Execute → Write-back (CDB) → Commit (in-order). Only issue and write-back are OOO; fetch/decode/rename/commit remain in order to maintain precise state. IPC = $w \times$ utilisation; utilisation depends on ILP, predictor accuracy and memory latency.

The front end (fetch/decode/rename) must deliver w instructions per cycle to keep the back end fed. A single mispredicted branch can empty the front end for 10–20 cycles in a deep pipeline, which is why branch prediction (Ch. 3) and a large instruction window (ROB+RS) are co-designed: bigger windows tolerate longer front-end stalls and longer memory latencies.



Remark 2.1 (Design intuition). Width is about how many you fetch; window size (ROB+RS+LSQ) is about how far you can look ahead. Performance is often window-limited before it is width-limited, especially on cache misses. Balancing the two under a power budget defines a microarchitecture generation.

2.11 Walk-Through: OOO Hides a Cache Miss

Consider: `LW x1,0(x2)` misses (10 cycles), `ADD x3,x4,x5` and `SUB x6,x7,x8` are independent, `ADD x9,x1,x10` needs the load. In-order stalls all after the LW for 10 cycles. OOO renames, dispatches all four, executes the two independent ALU ops while the load waits, and only the final ADD waits for the load's CDB broadcast. IPC $\approx 3/12 \approx 0.25$ in-order versus $\approx 3/11 \approx 0.27$? Actually the hide is small here; with a 64-entry window and many independent instructions the miss is fully hidden and IPC approaches width. This is why large windows matter for memory-bound code.

Cycle	In-order	OOO	Hidden?
1	LW (miss)	LW / ADD / SUB dispatched	—
2–10	stall	ADD, SUB complete on CDB	yes
11	ADD x9 waits	ADD x9 wakes and issues	—

The same mechanism also tolerates branch mispredicts when combined with speculation (Ch. 3): instructions after a predicted branch enter the ROB speculatively and are flushed on mispredict, wasting window slots but not correctness.

2.12 Key Takeaways

Superscalar provides width; OOO fills it by executing the next ready instruction. Renaming removes WAR/WAW so only RAW constrains scheduling. Tomasulo's RS+CDB is the archetype: distributed waiting, tag-based dataflow, and in-order commit for precision. Modern cores scale this with larger ROBs (200–500 entries), deeper LSQs, and aggressive memory speculation.

Forward link: width and OOO still stall on branches. Without predicting the next fetch address correctly, even a 4-wide OOO window empties. Chapter 3 adds the predictor, BTB, and speculation support that keep it full.

2.13 Exercises

Exercise 2.1. Sequence `MUL x1,x2,x3`; `ADD x4,x1,x5`; `SUB x6,x1,x7`; `ADD x1,x8,x9`.
Identify RAW/WAR/WAW. Show how renaming to physical registers removes WAR/WAW and draw the dataflow graph of true dependences.

Exercise 2.2. For a 2-way superscalar with one ALU and one LSU, schedule `LW x5,0(x1)`; `ADD x6,x5,x2`; `SUB x7,x8,x9`; `SW x7,4(x1)` for minimal cycles. Identify structural and data stalls and show which cycle each instruction issues and completes if OOO is allowed.

Exercise 2.3. Tomasulo RS has two entries; instructions need tags T1 (MUL, 4 cycles) and T2 (ADD, 1 cycle). ADD depends on T1. Trace RS state cycle by cycle from issue through CDB broadcast, showing Qj/Qk, Vj/Vk and when ADD wakes. Why does ADD wait even though its RS is allocated early?

Exercise 2.4. Explain why a single CDB becomes a bottleneck at width > 2. How do multiple CDBs or a bypass network help, and what new hazard arises for RS wake-up if two producers complete in the same cycle targeting the same consumer operand?

Exercise 2.5. A loop body has 6 independent ALU ops and 2 independent loads (all distinct addresses). With width 4 and infinite RS/ROB, estimate steady-state IPC. What limits it: width, dependences, or memory? How does LSQ disambiguation affect load issue?

Chapter 3

Branch Prediction, Speculation and BTB

Predict, fetch, speculate, recover: pay the price of being wrong rarely to win big when you are right.

3.1 Why Control Hazards Dominate

With forwarding, data hazards cost at most one cycle. Control hazards scale with pipeline depth: a 5-stage pipeline pays 1–2 cycles per mispredict, a 15-stage OOO superscalar pays 10–20 cycles of flushed work. Branches are frequent (15–25% of instructions) and their direction and target are unknown until late. The gap is filled by **prediction** (guess early) and **speculation** (execute speculatively, commit only if correct).

Without prediction, even perfect forwarding leaves CPI $\approx$ 1.2–1.5 on branch-heavy code. With a 95% accurate predictor that drops to $\approx$ 1.03–1.07, worth dozens of percent in performance.

Pipeline	Stages	Mispredict penalty c	CPI overhead ($b = 0.18, p = 0.95$)
Classic 5-stage	5	1–2	0.009–0.018
Modern 10-stage scalar	10	6–8	0.054–0.072
OOO deep (fetch–commit 15)	15	10–14	0.09–0.13

Deeper means more instructions in flight to flush, so prediction matters more as pipelines deepen: frequency scaling past $\approx 4\,\text{GHz}$ stopped partly because c grew faster than p could improve.

3.2 Branch Outcomes to Predict

A branch has two things to know: *direction* (taken/not-taken) and *target* (where to fetch next). Conditional branches (`BEQ, BNE, BLT, ...`) need both; unconditional jumps (`JAL, JALR`) have fixed taken direction but may need target prediction (especially `JALR` for returns).

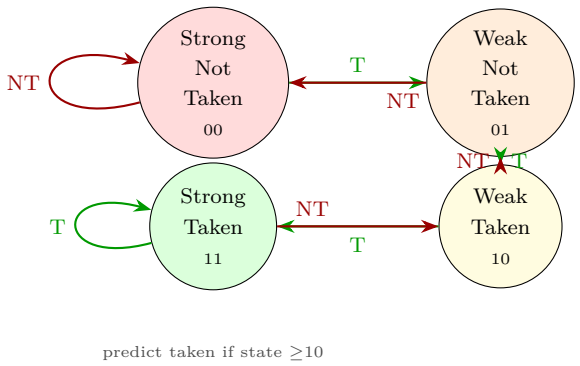
Prediction happens in fetch, before decode: given the PC, predict taken/not-taken and the next PC. Two structures cooperate: the **direction predictor** (counters, tables) and the **BTB** (branch target buffer) that remembers target addresses.

Type	Direction	Target	How predicted
Conditional (BEQ etc.)	taken/not	PC+imm vs PC+4	direction + BTB
Uncond. direct (JAL)	always taken	PC+imm	BTB only
Indirect (JALR)	always taken	register value	BTB / RAS / indirect predictor
Return (RET)	always taken	stack pop	RAS

An I-cache miss, a TLB miss, or a BTB miss can all stall fetch even with a perfect direction predictor: the frontend is a chain of small caches.

3.3 Bimodal and 2-Bit Saturating Counters

The simplest adaptive predictor indexes a table of 2-bit saturating counters by PC (or PC xored with history). The FSM has hysteresis: it takes two consecutive mispredicts to flip the prediction, which damps oscillation on loops.



A 1-bit counter mispredicts twice on a loop-closing branch (taken $n-1$ times, not taken once); the 2-bit counter mispredicts only once. Table size: 4K entries $\times 2$ bits $= 1\,\text{KB}$, 90–93% accurate on many workloads. Aliasing (different branches sharing a counter) degrades it, motivating two-level and gshare.

3.4 Two-Level, Gshare, Tournament and TAGE

Two-level / pattern history: keep a shift register of last h branch outcomes (global or per-branch), use it to index a second table. **Gshare**: XOR the PC with global history to index the counter table, mixing path and address.

Tournament: run two predictors in parallel (e.g., bimodal + gshare) and select the winner via a meta-counter updated on which was correct. **TAGE** (tagged geometric): multiple tables indexed with different history lengths, each entry tagged; the longest matching entry wins. TAGE with 20–60 KB dominates modern predictors at $> 96\%$ direction accuracy.

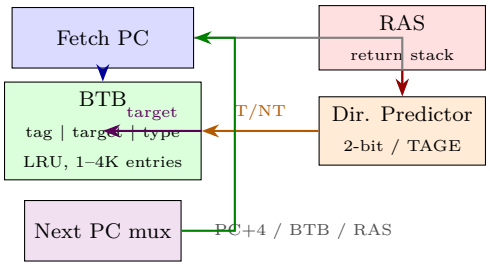
Static prediction (before any history is learned) uses heuristics: forward branches (loop exits) predicted not-taken, backward branches (loop backs) predicted taken, or the compiler can emit a taken hint bit. Modern

cores use static only for cold BTB misses; once the predictor warms, dynamic tables dominate. A cold branch that has been seen but whose counter is still weakly taken may still mispredict on its first taken occurrence, which is why Branch Target Buffers eagerly update on taken branches.

Example: pattern T,T,N,T,T,N,... needs $h = 2$ to learn that N follows T,T. A bimodal counter without history aliases all occurrences and mispredicts each N, while a GHR=11 predictor learns to use a different counter for the N case and can predict perfectly after warm-up. Correlated branches in e.g. `if (x<0) {...}; if (x<0 && y) {...}` are the textbook win for history-based predictors.

3.5 Branch Target Buffer and Return Address Stack

Knowing the direction is not enough; fetch needs the target address next cycle. The **BTB** is a cache keyed by branch PC, storing target PC, type, and sometimes fall-through. Look up BTB in parallel with the direction predictor: if hit and predicted taken, fetch from target; otherwise fetch PC+4 (or next sequential block for wide fetch).



Indirect branches (`JALR`, virtual calls, switches) have varying targets. The **Return Address Stack (RAS)** is a small hardware stack (8–32 entries) pushed on `CALL` and popped on `RET`, predicting returns with near-perfect accuracy. Indirect target predictors and Branch Target Address Caches (BTAC) handle the rest.

On a BTB hit and predicted taken, fetch redirects in the next cycle; on a miss, the branch is initially treated as not-taken (or target is computed after decode) costing one extra cycle even if direction was correct. BTB organisation matters: direct-mapped is small and fast but aliases; 2–4-way set-associative with LRU improves hit rate at the cost of parallel tag compares. Some cores fold the BTB into the L1 I-cache (branch decoded from predecode bits attached to each cache line) to save a separate array.

Fetch bandwidth for superscalar width w needs contiguous bytes. A taken branch in the middle of a fetch block wastes the tail, and a branch that spans a cache-line boundary may need two I-cache accesses. The instruction queue between fetch and decode smooths bubbles from BTB misses and from taken branches that were not predicted.

Case	Next PC	Cost if predicted correctly
Hit, predicted taken	BTB target	0 cycles (fetch continues)
Hit, predicted not-taken	PC + block	0 cycles
Miss, actually taken	target after decode	1–2 cycles redirect
Miss, actually not-taken	PC+4	0 cycles (lucky)
RAS hit for `RET`	RAS top	0 cycles; miss stalls BTB path

3.6 Speculative Execution and Recovery

A predicted branch is *speculative*: fetch, decode, rename and even execute instructions from the predicted path, but do not commit architectural state until the branch resolves. The ROB holds speculative results; a mispredict flushes all younger ROB entries, restores the RAT via checkpoints or a reorder stack, and redirects fetch. The deeper the speculation, the larger the flush penalty but also the larger the window for ILP.

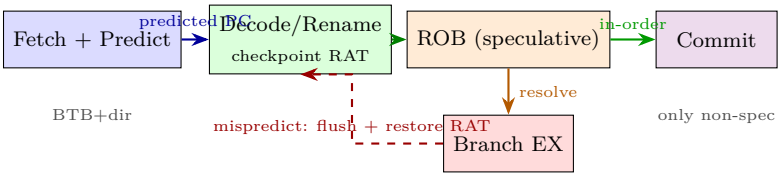
Listing 3.1: 2-bit predictor simulation (per branch)

```
state = 2  # 0 SN, 1 WN, 2 WT, 3 ST
def predict(): return state >= 2   # taken if WT or ST
def update(taken):
    global state
    if taken: state = min(3, state+1)
    else:     state = max(0, state-1)
# Loop: T,T,T,N repeats -> 1 mispredict per iteration
for taken in [True,True,True,False]*10:
    pred = predict()
    # mispredict = pred != taken
    update(taken)
```

Precise exceptions with speculation: exceptions are also deferred to commit, so a speculative instruction that page-faults down a wrong path never raises the fault if it is flushed. Meltdown/Spectre showed that microarchitectural side effects (caches, port contention) before the flush can leak speculative data, motivating speculation barriers, invisible speculation, and partitioning.

3.6.1 Speculative Pipeline and Flush Cost

In fetch, the predictor steers the next PC; decoded instructions carry a speculation tag and a snapshot of the RAT. When the branch resolves in EX (or later for complex branches), the tag is checked:



Flush cost c equals the number of stages between predict and resolve plus any ROB drain. With early resolve in ID, $c \approx 1$–2 (classic 5-stage); with OOO and late resolve, $c \approx 10$–15. Deeper speculation increases both the benefit (larger window) and the mispredict penalty.

Checkpoints avoid copying the whole RAT: on each branch a copy of RAT mappings and a ROB tail pointer are saved; on flush the checkpoint is restored in one cycle. Without checkpointing recovery would need to walk the ROB to undo mappings serially.

3.6.2 History-Based Correlation

A global history register (GHR) of the last h taken/not-taken bits captures correlation: `if(a) ...; if(b) ...` where b depends on a's path. Gshare xors PC with GHR to choose a counter, so the same branch sees different counters for different path histories. TAGE takes this further: tables T_0 (history 0), T_1 (history 4), T_2 (history 10), T_3 (history 25) each tagged; the longest matching tagged entry predicts. TAGE's geometric history gives both fast warm-up (short history) and long correlation capture.

Listing 3.2: Gshare index and BTB lookup (sketch)

```
GHR = 0  # h=8 bits
PHT = [2]*1024  # 2-bit counters, init weakly taken
BTB = {}  # pc -> target
def gshare_predict(pc):
    idx = (pc ^ GHR) & 0x3FF
    return PHT[idx] >= 2
def fetch(pc):
    if pc in BTB and gshare_predict(pc):
        return BTB[pc]  # predicted taken target
    return pc + 4
def update(pc, taken, target):
    idx = (pc ^ GHR) & 0x3FF
    PHT[idx] = min(3, PHT[idx]+1) if taken else max(0, PHT[idx]-1)
    if taken: BTB[pc] = target
    GHR = ((GHR << 1) | int(taken)) & 0xFF
```

3.6.3 Predictor Accuracy Landscape

Predictor	Storage	Accuracy (SPECint)	Warm-up
Always not-taken	0	$\approx 55\%$	none
Bimodal 4K×2b	1 KB	90–93%	100s branches
Gshare 8K×2b, $h = 12$	2 KB	93–95%	1000s
Tournament 2-level	4–8 KB	94–96%	longer
TAGE 32 KB	32 KB	96–97.5%	thousands

Accuracy numbers shift with workload: loops are easy, indirect branches and data-dependent conditions are hard. Hybrid predictors win on average because different branches favour different history lengths.

3.7 Performance: Putting Numbers Together

Let b be branch frequency, p accuracy, c flush cost. Then stall cycles per instruction $\approx b(1-p)c$. Example: $b = 0.18$, $p = 0.95$, $c = 12$ (deep OOO) $\Rightarrow$ 0.108 CPI overhead. At $p = 0.85$ overhead doubles to 0.32. Every 1% accuracy is ≈ 0.02 CPI on this model. BTB miss adds ≈ 1 cycle even when direction is correct because the target is unknown.

On a 3.5 GHz OOO core, a 12-cycle flush is ≈ 3.4 ns of wasted fetch bandwidth (≈ 40 instructions of width

4 flushed). Cutting c by early branch resolution (branch predictor + fast ALU) is as valuable as raising p, which is why Branch ALUs are placed early in the pipeline and why BTB and instruction prefetch are tightly integrated.

3.7.1 Window and Frontend Interaction

A large ROB (e.g., 224 entries) can hide a 14-cycle mispredict if the predictor is accurate enough: the window refills after the redirect before the oldest flushed instructions would have committed. If the predictor is weak, the window fills with wrong-path instructions that are then flushed, wasting energy and polluting caches. That is why predictor accuracy and ROB size are co-optimised: doubling ROB without improving p yields little gain on branchy code. Loop unrolling and if-conversion (replacing a hard-to-predict branch with predicated moves) reduce b, which helps regardless of p.

3.8 Key Takeaways

Control stalls dominate deep pipelines, so prediction is performance-critical. The 2-bit counter already captures hysteresis; gshare and TAGE add correlation and history length adaptation. BTB/RAS deliver the target; speculation executes ahead and uses ROB+RAT checkpoints to recover precisely on a mispredict. Accuracy and flush cost jointly set the control component of CPI.

Bridge: prediction hides control stalls, but the memory system still stalls the pipeline on every L1 miss. Chapter 4 adds coherence so multiple cores can share memory without stalling on every sharing miss, while keeping the programmer's view of memory sane.

3.9 Exercises

Exercise 3.1. Loop with 100 iterations (branch taken 99×, not taken once, then next loop). Compare 1-bit vs. 2-bit bimodal total mispredicts over 3 consecutive runs. Explain the extra mispredict in the 1-bit case.

Exercise 3.2. For a predictor with 1024 2-bit counters indexed by PC[11:2] (no tag), two branches at 0x1000 and 0x5000 alias to the same entry. Construct an interleaved execution order where they interfere and estimate extra mispredicts. How does gshare or a tagged table reduce aliasing?

Exercise 3.3. BTB has 512 entries, direct-mapped, $b = 0.20$, BTB miss rate 8%, miss costs 3 cycles (use target after decode) versus 1 cycle on hit with correct prediction. Compute extra CPI due to BTB misses. If a RAS handles 12% of branches (calls/returns) at 99% accuracy, recalc.

Exercise 3.4. Pipeline depth 16, mispredict penalty = depth-2 cycles. Branch frequency 22%, predictor P1 90% vs. P2 96%. Compute CPI control overhead for each and the speedup of P2 over P1. Relate to wall-clock for a 3 GHz core.

Exercise 3.5. Spectre v1: a speculative load down the wrong path brings a secret-dependent line into cache; later a probe leaks it even after the flush. Describe which structures must be partitioned or rolled back (BTB, caches, TLBs, port contention) to close the channel and the performance cost of each fix.

Chapter 4

Advanced Memory: Coherence and Consistency

Many caches, one memory: keep them coherent without strangling performance, and define what "correct" sharing means.

4.1 Why Coherence Matters

A chip has multiple cores, each with private L1/L2 caches sharing an address space. Without intervention, two cores can hold different values for the same address, violating the programmer's expectation of a single memory. **Coherence** ensures all cores eventually agree on each location's value; **consistency** defines when different locations' updates become visible and in what order.

Example failure without coherence:

Step	Core 0	Core 1
0	`x`=0 in DRAM, both caches empty	
1	write `x=1`, hits L1	reads `x` $\rightarrow$ 0 from DRAM (stale)
2	evicts dirty line later	still holds stale 0

Coherence protocols track who has a copy and who may write; consistency models restrict how cores may reorder loads and stores.

4.2 Coherence Properties

For a single address A, coherence enforces: (1) write propagation (a write eventually visible to all), (2) write serialization (all cores see writes to A in the same order), and (3) read returns the latest serialized write (before any later write overwrites it). Mechanisms are **snooping** (bus broadcast, all caches listen) or **directory** (a home node tracks sharers and forwards requests).

Write policy interacts: write-back caches keep dirty copies and must write back on eviction or when another core wants the data; write-through would simplify but costs bandwidth.

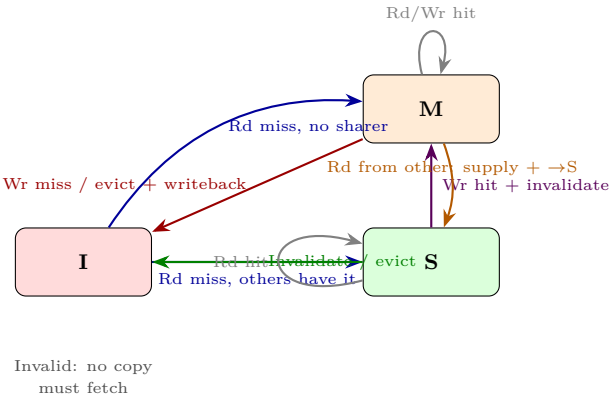
Message	Source	Effect
`BusRd`	cache on read miss	directory / snoop asks sharers; supplier provides data
`BusRdX`	cache on write miss	read-exclusive + invalidate sharers
`Invalidate`	writer in S→M	sharers move S→I, send Ack
`Ack / Data`	sharer / memory	completes the transition, may supply dirty data

A cache miss stalls the core (or at least that load) until the coherence transaction completes. Modern cores allow hits under miss (non-blocking caches, MSHRs) so other accesses to different lines proceed, but dependences on the missing line still stall.

Definition 4.1 (Stable vs. transient states)**.** Stable states are M/E/S/I as seen between transactions. Internally a line passes through transient states (e.g., `IS` waiting for data, `SM` waiting for acks) while messages are in flight. Verification must check that transient states never deadlock and that write serialization is preserved despite overlapping transactions.

4.3 MSI Protocol

The base three-state protocol attaches to each cache line one of: **M**odified (exclusive dirty, must write back), **S**hared (may be replicated, clean), **I**nvalid (no valid copy).



Events from the core (Rd, Wr) and from the bus (BusRd, BusRdX, Invalidate) drive transitions. On a write miss (`Wr miss`) the cache issues `BusRdX` (read-exclusive), invalidates sharers, and enters M. On a read miss where another core holds M, that core supplies the data and both end in S (or the supplier writes back).

MSI allows sharing for reads but a write requires invalidating all sharers first, costing bus traffic on every write to shared data.

4.3.1 MSI Walk-Through

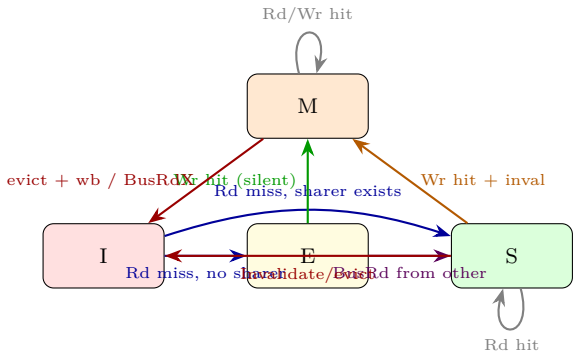
Initial: DRAM x=0, both caches I.

Step	Core action	Bus action	Core0	Core1	Data from
1	C0 Rd x	BusRd	S	I	DRAM
2	C1 Rd x	BusRd	S	S	LLC / C0
3	C0 Wr x=1	BusRdX (invalidate)	M	I	—
4	C1 Rd x	BusRd	S (downgrade, supply)	S	C0 cache-to-cache

At step 3 Core0 must collect ack from Core1 before completing; the store stalls in the store buffer until acks arrive. At step 4 Core0 supplies the dirty line and writes back; both end S with the new value. Repeating this with MESI would keep the line in E after step 1 (no sharer), so step 3 would be silent (E→M, no BusRdX).

4.4 MESI: Adding Exclusive

MSI cannot distinguish "shared clean with one copy" from "shared clean with many copies": a core that brings a line from DRAM to an empty system still enters S and must issue an invalidate on its first write, even though no sharer exists. MESI adds **E**xclusive (clean, sole copy): a line fetched with no other sharer enters E and can be silently upgraded to M on a write without bus traffic.

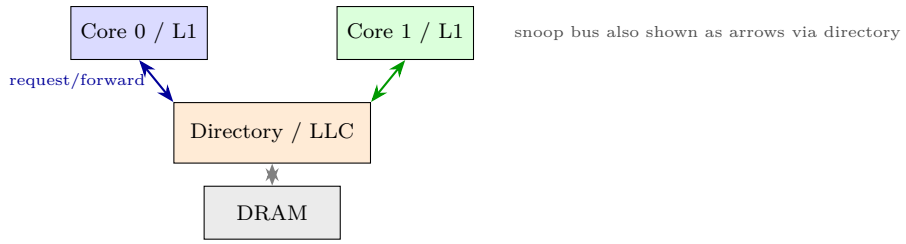


E is the performance win: private data and stack lines stay in E/M without coherence traffic. Transitions: E→S on snooped BusRd (now shared), E→M on `Wr hit` silently (no invalidate needed), M→I on eviction or when another core's `BusRdX` requests it (supply data and write back if needed).

MOESI (Adds **O**wned: dirty but shared, supplier without writeback) and MESIF (Adds **F**orward: designated responder) further optimise sharing of dirty lines and reduce broadcasts on read misses to shared data. Directory protocols scale similarly but the directory replaces bus snooping with point-to-point messages.

4.5 Snooping versus Directory

Snooping broadcasts every coherence request on a bus or ring; every cache checks its tags. Simple but bandwidth-limited to ≈ 8–16 cores. Directory keeps a per-line sharer vector at the home node (often LLC/ memory controller); requests go to the directory, which forwards only to sharers. Directory scales to hundreds of cores but adds an indirection hop and directory storage overhead.



4.6 False Sharing, Granularity and MOESI Extensions

Coherence acts on whole lines (64 B). Two independent variables on the same line cause **false sharing**: cores ping-pong the line between M states even though they touch disjoint bytes. Example: `int cnt[2]` with `cnt[0]` on Core0 and `cnt[1]` on Core1 still bounces every write because the line is one coherence unit. Fixes: pad structures to line boundaries, align thread-local data, or use per-core partitioning.

Extension	Extra state	Meaning	When it helps
MOESI	O (Owned)	dirty but shared, owner supplies without writeback	producer–consumer sharing
MESIF	F (Forward)	designated sharer to reply	many readers, one supplier
MESI	E (Exclusive)	clean sole copy	private/stack data

Owned (MOESI, AMD) lets a dirty line be shared without writing back: the owner (O) supplies data on snooped reads and remembers to write back on eviction. Forward (MESIF, Intel) picks one sharer as responder so multiple S copies do not all try to respond. Neither changes correctness, only traffic. Directory with MOESI can keep O in the LLC and serve reads without going to the owning core's L1.

Coherence granularity also affects bandwidth: a write to one byte still invalidates the whole line. Sector caches and write-combining buffers mitigate by merging multiple writes to the same line before issuing a single invalidate.

4.7 Coherence Performance

Misses to shared lines cost more than private misses: a read miss to an S line may be served from LLC (≈ 10 ns) but a read miss needing data from another core's M line costs a cache-to-cache transfer ($\approx$ 20–40 ns plus coherence messages). Invalidations are on the critical path for writes to shared data: the store cannot commit until all sharers acknowledge invalidation, which the LSQ and write buffer must track. Large sharing (many sharers) therefore stalls the writer longer.

A rule of thumb: minimise sharing, maximise private/E-state data, and align synchronisation variables (locks, flags) on their own lines away from payload data. Tools like false-sharing detectors sample coherence misses via performance counters (e.g., `MESI_Snoop_Hit`, `Offcore_Response`) to guide padding.

4.8 Memory Consistency

Coherence says each location is serialized; consistency says how different locations' accesses interleave as seen by different cores. Consider initial `x=y=0`:

Listing 4.1: Classic litmus test (store buffering)

```
# Core 0              # Core 1
SW  x1, 0(a0) # x=1    SW  x1, 0(a1) # y=1
LW  x2, 0(a1) # y=?    LW  x2, 0(a0) # x=?
# Under SC, at least one load sees 1. Under x86-TSO or weaker, both may see 0.
```

Sequential Consistency (SC): there exists a global interleaving of all cores' operations that respects each core's program order and where each load sees the latest store in that interleaving. Intuitive but expensive: it forbids most OOO and buffering optimisations.

Total Store Order (TSO, x86): loads may be reordered before earlier stores to different addresses (store buffer), but stores stay ordered with stores, loads with loads. The store-buffer litmus above is allowed to see (0,0); earlier litmus where both stores are seen is still forbidden.

Weak / release consistency (ARM, RISC-V) : almost any reordering is allowed except when the programmer inserts fences. RISC-V RVWMO defines `FENCE`, `acquire` and `release` annotations: a release store makes prior accesses visible before it, an acquire load makes later accesses start after it. Data-race-free programs with proper synchronisation appear SC; racy code has no guarantee.

Listing 4.2: Fences restore ordering in RISC-V

```
# Core 0: producer                    # Core 1: consumer
SW  x1, 0(a0)      # data=1             LW  t0, 0(a1)  # flag ?
FENCE  w, w                           FENCE  r, r
SW  x1, 0(a1)      # flag=1             LW  t1, 0(a0)  # data
# Without fences on weak model, consumer could see flag=1 but data=0.
# FENCE w,w orders stores; FENCE r,r orders loads.
```

Fences flush or order store buffers and drain load queues; they cost tens of cycles, so place them only at synchronisation points. Compilers map C++ `std::atomic` with `memory_order_release/acquire` to exactly these fences.

4.8.1 What Each Model Allows to Reorder

Reordering	SC	TSO	ARMv8	RVWMO
Store → later Load (diff addr)	No	Yes (SB)	Yes	Yes
Store → later Store	No	No	Yes*	Yes*
Load → later Load	No	No	Yes*	Yes*
Load → later Store	No	No	Yes*	Yes*

*Allowed unless ordered by `FENCE` or `acquire/release` annotations. SC allows none; TSO only relaxes Store→Load via store buffer; weak models relax all four.

The classic store-buffer allows core 0 to buffer `x=1` while loading `y` before the store is globally visible; both cores can then see (0,0). Under SC the store buffer must drain before the load, so at least one load sees 1. On RISC-V, a `FENCE RW,RW` between the store and load would restore SC for that pair. Acquire/release is lighter than a full fence: `AMOSWAP.aq` (acquire) orders later ops after it, `AMOSWAP.rl` (release) orders earlier ops before it, matching C++ atomics without a heavyweight `FENCE`.

A second litmus is *message passing*: `data=1; flag=1` on Core0, `if(flag) load data` on Core1. Without `FENCE W,W` after data and `FENCE R,R` before the data load, flag=1 does not imply data=1 on a weak model. With fences, or with `SW.rl flag` and `LW.aq flag`, the ordering is enforced.

4.9 Putting It Together: Coherence vs. Consistency

Coherence is per-location and enforced by MSI/MESI; consistency is cross-location and enforced by fences and store-buffer draining. A core with aggressive OOO may reorder a load past an earlier store (TSO-allowed) but must not reorder two stores to the same address (coherence violation). Modern cores speculate past fences and roll back on violation detection via the LSQ and coherence invalidations.

Speculative coherence adds nuance: a core may speculatively read a line in S, but if another core invalidates it before the speculative load commits, the LSQ must detect the invalidation and replay. Store buffers also interact: a store sits in the buffer until its line is in M/E; only then does it become globally visible. Draining the buffer is the consistency action; invalidating sharers is the coherence action.

4.10 Cache Optimisations Consolidated: Six Classic Tricks

Earlier chapters borrow one cache trick each in passing: out-of-order execution hiding a miss (Chapter 2), the fetch chain of I-cache, BTB and prefetch (Chapter 3), and coherence traffic for shared lines (Chapter 7). This section collects the six standard uniprocessor optimisations in one place.

Trick	What it buys
Victim cache	small fully associative buffer catches conflict-evicted lines
Stride/stream prefetcher	detects sequential or strided access, fetches ahead
Non-blocking (lockup-free)	MSHRs track misses so hits proceed underneath
Critical-word-first + early restart	requested word first, resume before line fills
TLB organisation	caches translations so hits avoid page walks
Sv39 interaction	3-level RISC-V tables set the walk cost on a TLB miss

Victim cache (Jouppi): a 4–16-entry fully associative buffer between L1 and the refill path holds just-evicted lines, turning conflict misses into fast victim hits without raising L1 associativity. **Hardware prefetchers** recognise streams and strides, such as the unit-stride sweeps of a stencil, and fetch lines before the core asks; accuracy matters because wrong-path prefetches waste bandwidth and pollute the cache. **Non-blocking (lockup-free) caches** attach Miss Status Holding Registers (MSHRs) to each outstanding miss so later hits

to other lines proceed underneath, which is exactly what lets the Chapter 2 out-of-order window keep draining during a miss. **Critical-word-first** delivers the requested word over the bus before the rest of the line, and **early restart** resumes the stalled load as soon as that word arrives rather than waiting for the full 64 B fill. The **TLB** caches virtual-to-physical translations so hits never walk the page table; on a miss under RISC-V **Sv39** (39-bit virtual address, three-level radix table) the walker may issue up to three dependent memory reads, each itself cacheable. Hence TLB reach (entries × page size, grown by superpages) must cover the working set, or page-walk traffic lands on the same interconnect analysed in Chapter 5.

4.11 Key Takeaways

MSI is the minimal coherent protocol; MESI's Exclusive state eliminates unnecessary invalidates for private data. Snooping versus directory is a scalability trade-off. Coherence is necessary but not sufficient: the consistency model (SC/TSO/RVWMO) specifies which cross-address reorderings the hardware may perform. Programmers rely on synchronisation (locks, fences, acquire/release) to restore intuitive ordering where it matters, paying fence cost only on those paths.

Remark 4.1 (Checklist). For any litmus test, ask: (1) is coherence violated (per-location order)? (2) is consistency violated (cross-location order)? (3) would a fence or an `.aq/.rl` fix it, and at what cost? Answering all three is the exam pattern for `flag/data` and store-buffer tests.

Next: with coherence and consistency in hand, Chapter 5 extends the hierarchy to LLC, interconnect and real SoC implications.

4.12 Exercises

Exercise 4.1. Two cores, line `A` initially I in both, DRAM=0. Trace MSI states cycle by cycle: Core0 Rd A, Core1 Rd A, Core0 Wr A, Core1 Rd A. List bus transactions (BusRd/BusRdX/invalidate), data supplier, and final states. Repeat for MESI and highlight where E avoids a transaction.

Exercise 4.2. For a snooping bus, explain why a write to a line in S must broadcast an invalidate even under TSO. What would break if the invalidation were deferred to the store buffer drain? Give a litmus test where deferral violates coherence.

Exercise 4.3. Store-buffer litmus from Listing 1: both cores see 0 under TSO but not under SC. Show a TSO execution with store buffers where both loads bypass their own buffered stores. Draw the global order that SC would require and why it cannot include (0,0).

Exercise 4.4. RISC-V weak model: thread 0 does `SW data=1; FENCE w,w; SW flag=1`, thread 1 does `LW flag; FENCE r,r; LW data`. Prove that if thread 1 sees `flag=1` it must see `data=1`. What happens without the fences? Give an interleaving where `flag=1, data=0` is visible.

Exercise 4.5. Directory vs. snooping: a 64-core chip runs a read-sharing workload (all cores read line `A` repeatedly). Compare bus traffic per read miss for snooping (broadcast to 63) vs. directory (ask directory, get data). Now consider a private-data workload (each core writes its own line). Which scales better and why does MESI E state matter more for private data?

Chapter 5

Interconnection Networks: Topology and Routing

How do thousands of cores, caches, and accelerators talk without drowning in wires?

5.1 Why Interconnects Matter

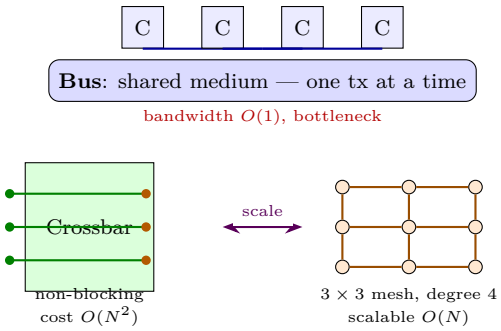
A single core is a fast worker on an isolated desk. A modern SoC or supercomputer is a city: 64–1000 cores, GPU tiles, memory controllers, and I/O must all exchange data. The streets and highways of this city are the **interconnection network** (IN). If the network stalls, cores idle—Amdahl's serial fraction is not code but wire.

Early machines used a single shared **bus**: one set of wires, one transaction at a time, snooped by all. Simple and coherent, but bandwidth is fixed and arbitration serialises every miss. At 4 cores it suffices; at 64 it collapses. The bus is a one-lane road into a metropolis.

We therefore replace the bus with scalable fabrics where many messages fly in parallel, trading simple broadcast for richer topology, routing, and flow control. The design space spans on-chip NoCs, off-chip supercomputer fabrics, and datacentre Clos networks—same principles, different cost constraints.

Definition 5.1 (Interconnection network metrics)**.** A topology is a graph (V, E) where V are nodes (cores/routers) and E are links. Key metrics: *diameter* D (max shortest path, worst latency), *bisection bandwidth* B_{bis} (min links cut to halve the network, peak all-to-all throughput), *degree* d (ports per router, cost), *average distance* $\bar{h}$ (mean hops), and *cost* (routers + links + wiring area).

Networks are judged by latency at low load, throughput at high load, and cost. A cheap ring wins on cost but loses on diameter; an ideal crossbar wins on performance but loses on cost ($O(N^2)$ switches). Real designs pick a point on this Pareto frontier.

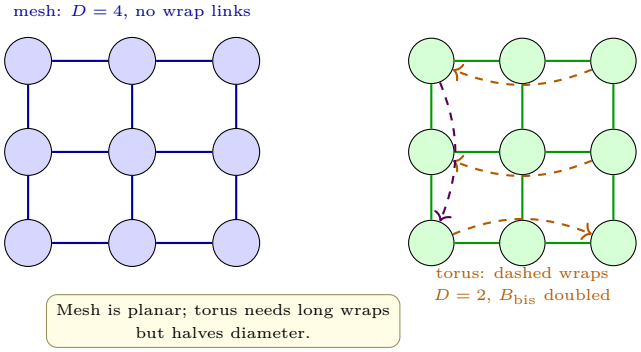


5.2 Topology Taxonomy

Direct networks: each node has an integrated router (mesh, torus, hypercube). **Indirect**: separate switch nodes connect terminals (crossbar, fat-tree, Clos). Direct suits on-chip (NoC) where wiring is planar; indirect dominates off-chip supercomputers where switch radix and cable length dominate.

- **Ring** (N nodes, degree 2, $D = \lfloor N/2 \rfloor$): cheapest, but diameter grows linearly. Bidirectional halves D.
- **2-D mesh** ($k \times k$, $N = k^2$, degree ≤ 4, $D = 2(k-1)$, $B_{\text{bis}} = k$): layout-friendly, VLSI planar. Intel mesh NoC, Tilera.
- **2-D torus**: mesh plus wrap-around links (degree 4, $D = k$, $B_{\text{bis}} = 2k$): doubles bisection at cost of long wraps.
- **Hypercube** ($N = 2^d$, degree d, $D = d$, $B_{\text{bis}} = N/2$): logarithmic diameter, but degree grows with N—wiring explodes.
- **Fat tree / Clos**: indirect, non-blocking with enough middle stages; $D = O(\log N)$, $B_{\text{bis}} = N/2$ at cost of many switches. Modern datacentres, NVLink, InfiniBand.

Meshes dominate chips because they map to 2-D silicon. At rack scale, fat-trees dominate because bisection matters more than wire length. Understanding this split explains why NoC papers show meshes and HPC papers show Clos.



5.3 Routing: From Determinism to Adaptivity

Routing picks the path a packet follows. Two independent decisions: *which* path and *when* to wait for links.

Dimension-order (DOR) on a mesh: go first in X, then Y (or X–Y). Simple, deadlock-free by construction, but oblivious to congestion—a hot spot on the X row stalls all traffic. **Adaptive routing** senses congestion (queue depth, credit count) and detours, improving throughput but risking deadlock, livelock (infinite wandering), and out-of-order delivery.

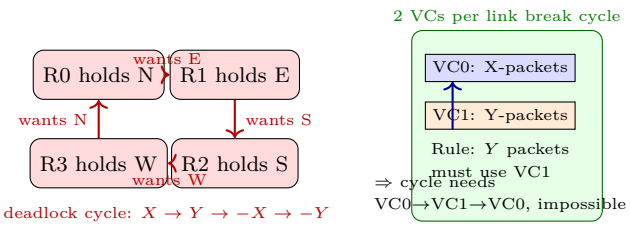
Deadlock: cyclic buffer wait—four packets each holding one link waiting for the next, none can advance. Classic on a 2×2 mesh with four packets circling clockwise. Broken by ordering resources: DOR's $X \to Y$ order forbids the $Y \to X$ turn that completes the cycle; or use **virtual channels (VCs)**—multiple logical queues per physical link—to break cycles without restricting turns. VCs also cure head-of-line blocking.

Flow control decides when flits move. **Store-and-forward**: whole packet buffered per hop (high latency, large buffers). **Wormhole**: packet split into *flits* (flow-control units), flits pipeline across routers; header opens the path, body follows; stalled header blocks body flits strung across routers (small buffers, low latency, but head-of-line blocking). **Virtual-cut-through** is the middle ground: forward as soon as next router has room for the whole packet.

Listing 5.1: Dimension-order routing (X-then-Y) on a $k \times k$ mesh.

```python
def dor_next_hop(src, dst, k):
    sx, sy = src % k, src // k
    dx, dy = dst % k, dst // k
    x, y = sx, sy
    path = [src]
    while x != dx:  # X dimension first
        x += 1 if dx > x else -1
        path.append(y * k + x)
    while y != dy:  # then Y dimension
        y += 1 if dy > y else -1
        path.append(y * k + x)
    return path  # e.g. dor_next_hop(0, 8, 3) -> [0,1,2,5,8]

# Adaptive variant: pick less-congested minimal direction
def adaptive_next(src, dst, congest):
    cand = minimal_directions(src, dst)  # up to 2 options
    return min(cand, key=lambda n: congest[n])
```



5.4 NoC Router and Flow Control

A canonical NoC router has five ports (N/S/E/W + local), each with VC buffers, a route-compute stage, VC allocator, switch allocator, and crossbar. Pipeline: RC → VA → SA → ST (switch traversal) → LT (link traversal). Speculative allocation collapses VA+SA for low load. Credit-based flow control keeps producer from overrunning the next router: downstream advertises free flit slots; upstream sends only when credits remain. This is lossless backpressure, essential for coherence messages that cannot be dropped.

Power and area of the router matter: at 64 cores the NoC can be 15–25% of chip area. Optimisations include bufferless deflection routing (drop buffering, deflect on conflict), concentrated meshes (4 cores share one router), and express virtual channels (bypass intermediate routers).

5.5 Performance: Latency, Throughput, Contention

Zero-load latency $T_0 = H \cdot t_r + D/v + L/B$: hops × router delay plus wire propagation plus serialisation (L bytes at B bytes/s). Under load, contention adds queueing—modelled as $M/D/1$ per link, so latency rises steeply after ∼50% of saturation throughput.

Bisection bandwidth caps global throughput. For $k \times k$ mesh, $B_{\text{bis}} = k$ links × link BW. Doubling k quadruples nodes but only doubles bisection—hence meshes are bisection-limited at scale; torus and fat-trees trade cost for bisection. Uniform random traffic saturates at $\approx B_{\text{bis}}/(\bar{h})$; adversarial (transpose, hotspot) saturates far earlier, exposing the topology's weakness.

Wormhole's head-of-line blocking is eased by VCs: each VC has its own buffer, so a stalled packet in VC0 does not block VC1 on the same wire. More VCs improve throughput but add allocator complexity and power. Modern NoCs use 2–4 VCs, enough to separate coherence classes (request/response) and break protocol deadlock.

Listing 5.2: Wormhole flit pipeline: latency vs. store-and-forward.

```
def wormhole_latency(hops, t_r, t_w, flits, bw):
    # t_r: router pipeline per hop, t_w: wire, flits pipeline
    header = hops * (t_r + t_w)
    body   = (flits - 1) / bw  # flits stream behind header
    return header + body

def store_forward_latency(hops, pkt_bytes, bw, t_r):
    return hops * (pkt_bytes / bw + t_r)

print(wormhole_latency(4, 2, 1, 8, 1))   # ~12 + 7 = 19 cycles
print(store_forward_latency(4, 8, 1, 2)) # 4*(8+2)=40 cycles
# wormhole halves latency for multi-flit packets
```

5.6 Exercises

Exercise 5.1. Bus vs. mesh: a 64-core chip runs coherence broadcasts each miss. Why does a bus saturate? Estimate bus transactions/s if each miss moves 64 B at 20 GB/s link BW, and compare to a 8×8 mesh with $B_{\text{bis}} = 8$ links. When is a bus still preferable?

Exercise 5.2. For $k = 8$ ($N = 64$): compute diameter and bisection bandwidth for ring, 8×8 mesh, 8×8 torus, and 64-node hypercube ($d = 6$). Rank them and explain which metric matters for all-to-all vs. nearest-neighbour traffic.

Exercise 5.3. Show that X-Y DOR on a mesh is deadlock-free by ordering resources. Construct a 4-node cycle that would deadlock with unrestricted minimal routing, and explain how 2 VCs (escape VC) break it without forbidding turns.

Exercise 5.4. Wormhole vs. store-and-forward: a 5-flit packet traverses 6 hops; $t_r = 3$, $t_w = 1$ cycle, link BW = 1 flit/cycle. Compute both latencies. If one link stalls for 10 cycles, how far does the wormhole packet "stretch" and which routers' buffers fill?

Exercise 5.5. Design a fat-tree for $N = 16$ terminals with 4-port switches. How many switches per level? What bisection bandwidth does it provide vs. a 4×4 mesh? Why do datacentres prefer fat-trees despite higher switch count?

Chapter 6

Accelerators: GPUs, TPUs, and Domain-Specific Architectures

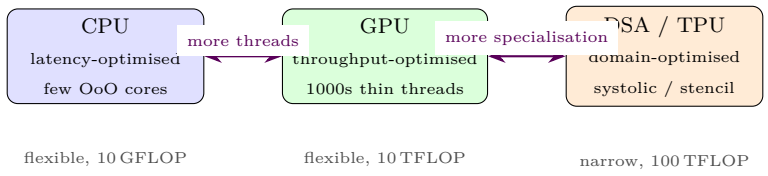
When general-purpose hits the wall, specialise: trade flexibility for 10–100× efficiency.

6.1 Why Accelerate? The End of Free Lunch

For decades Dennard scaling and Moore's law gave faster, cooler transistors for free. Around 2005–2010 both stalled: we can still add transistors but cannot power them all at once (dark silicon) and single-thread performance flatlined. The response is heterogeneity: keep a few big out-of-order cores for sequential code and add **domain-specific accelerators** (DSAs) that do one thing extremely well.

An accelerator wins by matching structure to workload: massive parallelism for graphics/ML, systolic dataflow for matrix multiply, fixed-function pipelines for video/crypto. The price is programmability and Amdahl's law: if only fraction p is acceleratable, overall speedup $\leq 1/(1-p)$.

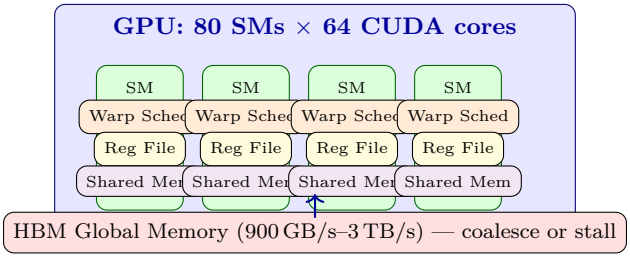
Definition 6.1 (Domain-specific architecture (DSA))**.** Hardware specialised to a domain (ML, graphics, DSP) that sacrifices generality for order-of-magnitude gains in performance, performance/Watt, or performance/mm^2 on that domain, while remaining programmable within it.



6.2 GPUs: SIMT at Scale

A GPU is not a vector machine but **SIMT** (single-instruction, multiple-thread): thousands of threads execute the same kernel, grouped into *warps* (32 threads on NVIDIA) that advance in lockstep. Divergence (branch where threads disagree) serialises the warp—a key performance pitfall.

Hierarchy: *Streaming Multiprocessor (SM)* → warps → threads. Each SM has register files, shared memory (scratchpad), L1, and many CUDA cores / tensor cores. Global DRAM bandwidth is critical (HBM 1–3 TB/s); caches help but scratchpad is software-managed. Memory coalescing (warp accesses contiguous addresses) determines whether you get 90% or 10% of BW.



A canonical CUDA kernel launches a grid of blocks; each block maps to an SM, threads within a block share scratchpad and can barrier-sync. Performance hinges on occupancy (enough warps to hide DRAM latency), coalescing, and avoiding bank conflicts in shared memory.

Listing 6.1: CUDA vector add: SIMT kernel, grid of threads.

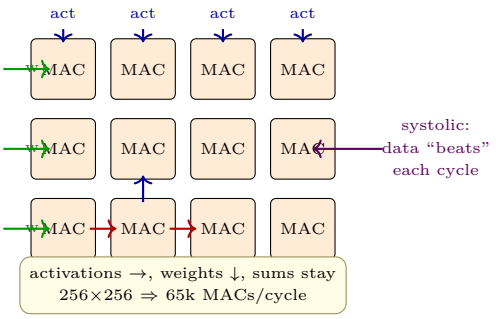
```
__global__ void vecAdd(float *a, float *b, float *c, int n){
    int i = blockIdx.x * blockDim.x + threadIdx.x; // global thread id
    if(i < n) c[i] = a[i] + b[i];                  // one thread per element
}
int main(){
    int n = 1<<20; float *a,*b,*c; // device pointers
    cudaMalloc(&a, n*sizeof(float)); cudaMalloc(&b, n*sizeof(float));
    cudaMalloc(&c, n*sizeof(float));
    int blk = 256; int grid = (n + blk - 1)/blk;
    vecAdd<<<grid, blk>>>(a,b,c,n); // launch: grid*blk threads
    cudaDeviceSynchronize();
}
```

Divergence example: `if(threadIdx.x%2) {...}` splits every warp into two passes, halving throughput. Sorting work by branch outcome before launch recovers performance—a reminder that SIMT is not free MIMD.

6.3 TPUs and Systolic Arrays

Google's TPU targets dense matrix multiply $C+ = A \times B$, the heart of DNNs. It uses a **systolic array**: a 2-D grid of MAC units where data flows rhythmically (systole) each cycle—weights flow down, activations flow right, partial sums accumulate in place. No fetch/decode per operation; data reuse is structural.

A 256×256 TPU v1 array does 65k MACs/cycle at low precision (int8/bf16). Weights are preloaded, then activations stream: each weight participates in many computations without revisiting DRAM. This exploits the roofline: AI of matmul is high ($O(n)$), so compute-bound design wins.



TPU also has large software-managed buffers (24 MiB), deterministic scheduling (no caches), and reduced precision—system co-design from model to silicon. Later TPUs add sparsity support and interconnect (ICI 3-D torus) for distributed training.

6.4 Designing a DSA: Roofline and Specialisation

Not every kernel deserves an accelerator. The roofline model guides the decision: if a kernel's arithmetic intensity (AI = ops/byte) sits far below the ridge, it is memory-bound and a compute monster will starve; improving BW or locality helps more. If AI is high (matmul, convolution), adding MACs moves the roof up linearly.

DSA principles (Hennessy & Patterson): (1) lean into the domain's dominant kernel (2) exploit its parallelism pattern (data-parallel, systolic) (3) minimise data movement (buffers near compute, compressed formats) (4) trade precision for efficiency (int8/bf16) and (5) co-design the compiler/scheduler to keep the array busy. A well-designed DSA moves both roofs: higher peak via specialised units and higher effective BW via locality.

Example: sparse DNNs are irregular; a dense systolic array wastes cycles on zeros. A sparse-aware DSA adds zero-skipping, load-balancing, and compressed sparse formats— gaining 2–4× even though control is more complex. The art is choosing which irregularity to accelerate and which to leave on the general core.

6.5 Chisel: Hardware Construction Language

Writing accelerators in Verilog is verbose and unparameterised. **Chisel** (Scala-embedded HDL) lets you write *generators*: parameterised, type-safe hardware that elaborates to Verilog. A Chisel `Module` describes registers and combinational logic; Scala loops generate replicated hardware at elaboration time, not runtime.

Chisel's strength is agile DSA design: one generator produces a family of accelerators (different bitwidths, array sizes) with formal testing in Scala. It powers RISC-V cores (Rocket, BOOM) and many ML accelerators. Libraries like Chipyard compose cores, accelerators, and NoCs from generators. The trade is learning Scala and trusting the elaboration chain to emit correct Verilog; verification remains essential.

Listing 6.2: Chisel: parameterised adder and systolic PE. Elaborates to Verilog.

```
import chisel3._
import chisel3.util._

// Parameterised N-bit adder (generates N full-adders)
```

```
class ParamAdder(val N: Int) extends Module {
  val io = IO(new Bundle{
    val a   = Input(UInt(N.W)); val b = Input(UInt(N.W))
    val cin = Input(Bool());    val sum = Output(UInt(N.W))
    val cout= Output(Bool())
  })
  val sum = io.a +& io.b + io.cin.asUInt
  io.sum := sum(N-1,0); io.cout := sum(N)
}

// Systolic processing element: weight-stationary
class PE(val W: Int) extends Module {
  val io = IO(new Bundle{
    val a_in = Input(SInt(W.W)); val b_in = Input(SInt(W.W))
    val a_out= Output(SInt(W.W)); val b_out= Output(SInt(W.W))
    val c    = Output(SInt(W.W))
  })
  val wReg = RegNext(io.b_in)          // weight stays
  val psum = RegInit(0.S(W.W))
  psum := psum + io.a_in * wReg
  io.a_out := RegNext(io.a_in); io.b_out := wReg; io.c := psum
}
```

Modern GPUs also add **Tensor Cores**: 4×4 matrix-multiply units that fuse many MACs into one instruction, boosting bf16/fp8 throughput 8–16× over CUDA cores. Mixed-precision training uses fp16/bf8 for matmul and fp32 for accumulation, exploiting the TPU lesson inside a GPU.

At system scale, multi-GPU nodes use NVLink/NVSwitch (300–900 GB/s) in a fat-tree or full mesh to keep data parallel training from becoming interconnect-bound. The same topology/routing lessons from Ch. 5 reappear at rack scale.

Together, GPUs teach SIMT throughput, Tensor Cores teach mixed-precision dataflow, TPUs teach systolic specialisation, and Chisel teaches how modern DSAs are productively built—the pillars of heterogeneous architecture.

6.6 Exercises

Exercise 6.1. A kernel is 70% parallelisable and runs on a 2048-core GPU at 80% efficiency for the parallel part. What overall speedup vs. a single core? How does divergence that halves warp throughput change the answer? Relate to Amdahl.

Exercise 6.2. Explain SIMT vs. SIMD vs. MIMD. Why does `if(tid%2) {A} else {B}` hurt a GPU but not a multicore CPU? Propose a code transform to recover performance.

Exercise 6.3. A 128×128 systolic array at 700 MHz does 1 MAC/cycle/PE. What peak int8 TOPS? If DRAM BW is 900 GB/s, what arithmetic intensity is needed to stay compute-bound? Why does weight-stationary dataflow help?

Exercise 6.4. Contrast GPU scratchpad (shared memory) and CPU cache: who manages them, when does each win, and what happens on bank conflicts or cache misses? Give a tiling example.

Exercise 6.5. Write a Chisel generator for an $N \times N$ systolic array using the `PE` above. How do you parameterise bitwidth W and size N? What Scala construct replicates PEs, and why does this beat hand-written Verilog for DSA exploration?

Chapter 7

Parallelism: Multicore, Coherence, and Synchronisation

Many cores, one memory: how to share correctly and quickly.

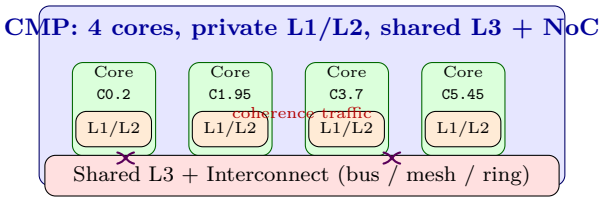
7.1 From One Core to Many: Why Multicore?

Frequency scaling hit a power wall: $P \propto CV^2 f$ and Pollack's rule says single-core performance grows as $\sqrt{\text{area}}$—doubling area gives only $\sim$40% more speed. The industry answer was to duplicate cores: a chip-multiprocessor (CMP) reuses the same core design N times and lets software exploit thread-level parallelism (TLP). Throughput then scales near-linearly if work parallelises, at the cost of making shared memory correct.

Shared memory is intuitive: all cores see one address space, communication via loads/stores. The hardware must make this illusion efficient while preserving correctness. Two pillars do that: **cache coherence** (one value per address) and **memory consistency** (order of values across addresses). Get them wrong and programs see stale data or impossible reorderings.

Definition 7.1 (Cache coherence vs. consistency)**.** *Coherence* guarantees that writes to a single address become visible to all cores in a total order (eventually every read sees the last write). *Consistency* defines the order in which writes to *different* addresses become visible—the memory model (SC, TSO, weak).

Without coherence, core 0 could read $x = 0$ from its cache while core 1 has just written $x = 1$ to its own cache—the system would violate the single-memory illusion. Every private cache therefore participates in a coherence protocol.

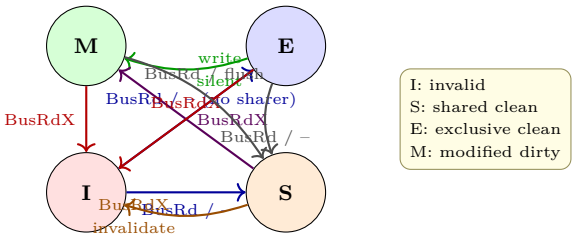


7.2 Cache Coherence: MSI and MESI

The classic protocol is **MSI**: each block is Modified (dirty, exclusive), Shared (clean, may be in many caches), or Invalid (not present). Stable states plus transient states handle races. On a read miss, the cache issues `BusRd`; if another cache holds M, it writes back and moves to S. On a write miss/hit to S, it issues `BusRdX` (read-exclusive), invalidating all sharers.

MSI wastes traffic on first reads that will later be written (read then upgrade). **MESI** adds Exclusive (clean, exclusive): a block loaded unshared enters E, and a later write can silently upgrade E→M without bus traffic. This common case (private data) is why all real machines use MESI/MOESI/MESIF variants.

Two enforcement styles: **snooping** (all caches watch a bus/NoC broadcast, $O(N)$ broadcasts, simple, needs ordered interconnect) and **directory** (home node tracks sharer list, point-to-point messages, scalable to many cores but indirection latency). Modern chips snoop within a cluster and use directories between clusters.



Coherence adds traffic and latency: a miss may need to invalidate k sharers and wait for acknowledgements. False sharing—two unrelated variables in the same 64 B line bouncing between cores—amplifies this. Padding or per-core batching fixes it.

7.3 Consistency: What Order Do Writes Become Visible?

Coherence says nothing about ordering across addresses. Consider Dekker's flag protocol: core 0 does $x = 1$; $r1 = y$ and core 1 does $y = 1$; $r2 = x$. Under **sequential consistency (SC)**—"all ops appear in one global order respecting each core's program order"—$r1 = r2 = 0$ is impossible (one write must precede the other's read). But modern machines (x86 TSO, ARM weak) reorder to hide store latency with store buffers: both reads can see 0, breaking Dekker.

x86 is TSO (total store order): loads bypass earlier stores to different addresses, but stores are ordered and loads are ordered. ARM/RISC-V are weaker: almost any reordering is allowed unless a `fence` is inserted. Compilers and programmers use acquire/release annotations; hardware reorders freely underneath, gaining 10–30% performance while correctness is recovered by fences only where needed. The RISC-V memory model even maps `acquire` to `fence r,rw`.

Understanding the memory model matters: 90% of "mysterious" concurrency bugs are missing fences or misunderstood atomics, not coherence. Tools like herd and litmus tests let architects verify the model exhaustively.

7.4 Directory Protocol and Scaling

Snooping broadcasts to all cores—$O(N)$ messages per miss—which is untenable beyond ~32 cores. A **directory** stores, per block, the sharer bit-vector at the home node. On a write, the writer contacts the directory, which point-to-point invalidates only sharers. Cost is indirection (request → directory → sharers → ack) and directory storage (N bits per block). Sparse directories spill to memory; coarse vectors trade false invalidations for smaller tags.

Modern hierarchies are hierarchical: snooping inside a 4–8 core cluster (low latency) and directory between clusters/chiplets (scalable). AMD Zen and Intel Sapphire Rapids use a mix—coherence is not one protocol but a family tuned to distance.

7.5 Synchronisation and Atomic Primitives

Cores synchronise with **atomic read-modify-write** ops: `test-and-set`, `fetch-and-add`, `compare-and-swap (CAS)`. Locks built from them need care: a naive spinlock with `CAS` hammers the interconnect. Better: `test-and-test-and-set` (spin on cached copy, CAS only when free) or queue locks (MCS) where each waiter spins on its own line, eliminating invalidation storms.

Barriers, condition variables, and lock-free data structures all reduce to these primitives plus fences. Performance pitfalls: contention (many cores on one lock serialise), convoying, and priority inversion.

Listing 7.1: Spinlocks: naive vs. test-and-test-and-set and MCS queue lock sketch.

```
// Naive: CAS every iteration -- storms the bus
void spinlock_naive(int *lock){ while(__sync_lock_test_and_set(lock,1)); }
// Better: spin on cached S copy, CAS only when it looks free
void spinlock_ttas(int *lock){
    while(1){
        while(*lock==1) __asm__ volatile("pause" ::: "memory"); // spin S
        if(__sync_lock_test_and_set(lock,1)==0) break;          // try M
    }
}
// MCS: each thread spins on its own qnode -- no invalidation storm
typedef struct qnode{ struct qnode *next; int locked; } qnode;
void mcs_lock(qnode **L, qnode *me){ me->next=NULL; me->locked=1;
    qnode *pred=__sync_lock_test_and_set(L, me);
    if(pred){ me->locked=1; pred->next=me; while(me->locked); }
}
```

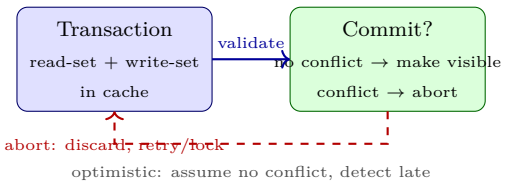
7.6 Transactional Memory

Locking is coarse, deadlock-prone, and inhibits composability. **Transactional memory (TM)** lets a block execute atomically: either all its memory ops commit or none do. Hardware TM (HTM, e.g., Haswell TSX) tracks read/write sets in caches; on conflict or overflow it aborts and falls back. Software TM (STM) instruments loads/stores.

Transactions compose: two atomic blocks inside a transaction remain atomic together, unlike two locks which can deadlock. The challenge is forward progress (livelock on repeated aborts) and I/O (cannot roll back). Real systems use TM for optimistic concurrency: try a transaction, on abort fall back to a lock.

Listing 7.2: Transactional increment: HTM intrinsic vs. lock fallback.

```
int counter = 0; int lock = 0;
void inc_htm(void){
    if(_xbegin()==_XBEGIN_STARTED){ // Intel TSX
        if(lock==0){ counter++; _xend(); return; }
        _xabort(0);
    }
    // fallback path: acquire lock
    while(__sync_lock_test_and_set(&lock,1));
    counter++;
    __sync_lock_release(&lock);
}
// STM equivalent: __transaction_atomic { counter++; }
```



Coherence, consistency, synchronisation, and TM are one continuum: coherence moves data, consistency orders it, synchronisation enforces ordering intentionally, and TM makes ordering speculative and composable. A well-tuned parallel program balances all four: minimise sharing and false sharing for coherence, insert fences only where the memory model demands for consistency, choose scalable locks/barriers for synchronisation, and use TM where optimistic concurrency beats pessimistic locking.

Putting it together, a single contended counter can bottleneck an entire application. Batch per-core counts locally, flush periodically under a coarse lock or transaction, and throughput scales—a pattern seen from Linux kernel counters to database logs.

7.7 Exercises

Exercise 7.1. Two cores share $x = 0$. Core 0 writes $x = 1$, core 1 reads x immediately after. Under MSI, trace messages (BusRdX, invalidations, writeback) for write-back caches. How many bus transactions? How does MESI's E state save one?

Exercise 7.2. Explain why `test-and-set` spinlock storms the interconnect while `test-and-test-and-set` does not, using M/S states. How does MCS queue lock further reduce traffic to $O(1)$ per handover? Sketch the cache-state sequence.

Exercise 7.3. Give the Dekker/litmus test where SC forbids $r1 = r2 = 0$ but TSO allows it via store buffers. Draw the store-buffer diagram and show where a `fence` restores SC. Why do ARM/RISC-V need fences even more?

Exercise 7.4. False sharing: an array `int a[64]` holds per-thread counters with one int per thread, 16 threads. Why does performance collapse vs. padding each counter to 64 B? Quantify invalidations per increment and propose a fix.

Exercise 7.5. HTM transaction reads A, B and writes C. Another thread writes A concurrently. What read/write sets conflict, and when is the abort detected (eager vs. lazy)? Why must a transaction that does I/O fall back to a lock, and what is the livelock risk if all threads keep aborting?

Chapter 8

Performance, Roofline, and the Future

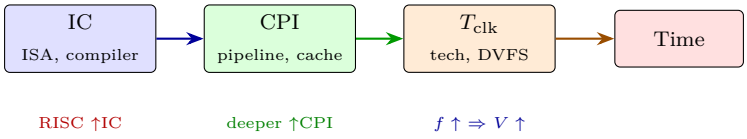
How fast, how efficient, and what comes after Moore?

8.1 Measuring Performance: Latency, Throughput, Speedup

Performance has two faces: **latency** (time per task, what the user feels) and **throughput** (tasks per unit time, what the datacentre sells). They can diverge: pipelining improves throughput while slightly worsening latency due to hazards and registers. Always state which you optimise.

The Iron Law still rules: CPU time $= IC \times CPI \times T_{\text{clk}} = IC \times CPI/f$. ISA sets IC, microarchitecture sets CPI and f, technology sets T_{clk} and power. A single optimisation that halves IC but doubles CPI may not win; only the product matters.

Speedup of B over A is $S = T_A/T_B$. "30% faster" means $S = 1.30$ ($T_B = T_A/1.30 \approx 0.77\,T_A$), not $T_B = 0.70\,T_A$. Sloppy phrasing inverts the ratio; always report the direction and baseline. For parallel machines, strong scaling (fixed N) follows Amdahl, weak scaling ($N \propto P$) follows Gustafson—both are true under different scaling assumptions, and confusing them misleads.

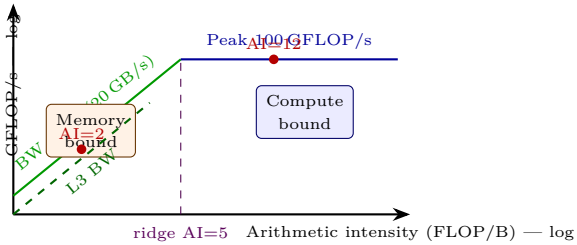


8.2 Amdahl, Gustafson, and the Roofline Model

If fraction p is accelerated by $s\times$ and $1-p$ is not: $S_{\text{Amdahl}} = 1/((1-p) + p/s) \leq 1/(1-p)$. The serial fraction dominates. Doubling s from 10 to 20 when $p = 0.8$ moves S from 3.57 to only 4.00—diminishing returns are brutal. The lesson is to attack the common case, then attack the remaining serial fraction.

Gustafson reframes: if problem size scales with the machine, the parallel fraction grows. Scaled speedup $S_{\text{scaled}} = (1-p) + p \cdot s$. With $p = 0.8$, $s = 10$: $S_{\text{scaled}} = 8.2$ vs. 3.57 fixed-size. Real users run larger problems on bigger machines, so both laws matter—strong vs. weak scaling.

The **Roofline model** unifies compute and memory: Attainable = min(Peak FLOPS, BW × AI), where AI = FLOPs/byte from DRAM. On a log-log plot the slanted part is memory-bound, the flat part compute-bound, the knee (ridge) is where Peak = BW × AI. Mark each kernel on the roofline: optimising locality tiles it right (higher AI); unrolling/SIMD pushes it up.



Modern roofs have multiple ceilings: DRAM, L3, L2 BW each give a slanted line, and compute ceilings differ without/with FMA, without/with Tensor Cores. A kernel may be L3-bound but DRAM-unbound—blocking to fit in L3 moves it from the DRAM roof to the L3 roof.

Listing 8.1: Roofline classification and attainable performance.

```
peak = 100   # GFLOP/s
bw_dram = 20 # GB/s
bw_l3 = 200  # GB/s (higher slope)
ridge = peak / bw_dram  # 5 FLOP/B

def classify(ai):
    attainable = min(peak, bw_dram * ai)
    bound = "memory" if ai < ridge else "compute"
    return bound, attainable

for ai in [1.5, 2, 12]:
    print(ai, classify(ai))
# 1.5 -> memory 30 GF/s; 12 -> compute 100 GF/s
# Tiling 1.5->6 moves from memory to compute: 30->100 GF/s win

def speedup_amdahl(p, s): return 1/((1-p)+p/s)
print(speedup_amdahl(0.8, 10))  # 3.57
print(speedup_amdahl(0.8, 20))  # 4.00 -- diminishing
```

8.3 Benchmarking: Pitfalls and Discipline

A benchmark must represent real use. Suites: SPEC CPU (single-thread), PARSEC (multicore), MLPerf (training/inference), TPC-C (transactions). Pitfalls: cherry-picking one Favourable program, cache-resident inputs that hide DRAM, comparing `-O0` vs. `-O3`, turbo on vs. off undisclosed, one run with no variance, and "benchmarketing" tuning only for the benchmark.

Report geometric mean for ratios (SPEC ratio = $T_{\text{ref}}/T_{\text{measured}}$; suite score = $\sqrt[n]{\prod r_i}$) because arithmetic mean breaks ratio semantics. Report median, std dev, and 95% CI across $\geq$5 runs, pin threads, warm caches realistically, and disclose flags. A speedup over an unoptimised baseline is not a speedup—optimise the baseline

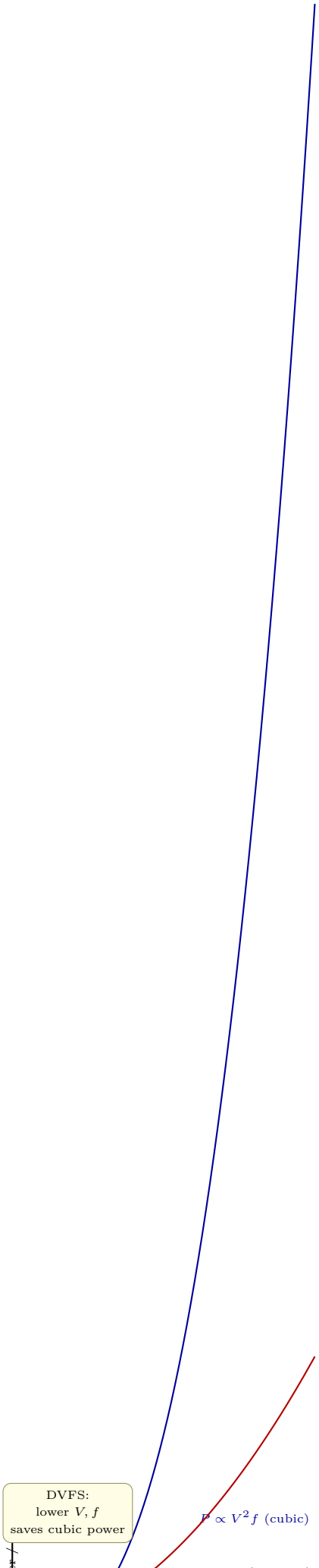
first. Statistical rigour matters: a single outlier (GC pause, thermal throttle) can dominate a mean; use box plots for distributions.

Example of the arithmetic-mean trap: two programs where machine A ratios are $(10, 0.1)$ and B are $(1, 1)$. Arithmetic means both ≈5.05 vs. 1.0—A looks faster despite being 10× slower on one program. Geometric means: A $\sqrt{1} = 1.0$, B 1.0—correctly tied and B is more consistent.

8.4 Energy, Power, and Dark Silicon

Energy = power × time (Joules); power = energy/time (Watts). Datacentre cost is power (cooling, provisioning) plus energy (electricity). Metrics: performance/Watt (FLOPS/W) and energy-delay product EDP = energy × time (trades latency vs. joules). $P_{\text{dynamic}} = \alpha C V^2 f$ dominates; P_{static} grows with temperature and small nodes.

DVFS trades f and V cubically: lowering f and V saves $V^2 f$ power at linear time cost, improving energy but hurting EDP if voltage cannot drop. Race-to-sleep (burst at high f, then power-gate) vs. slow-and-steady depends on static power. Dark silicon—we cannot power all transistors at max f—forces heterogeneity: keep the best accelerator powered, gate the rest.


DVFS:
lower V, f
saves cubic power
$P \propto V^2 f$ (cubic)

8.5 The Future: Chiplets, 3-D, Near-Memory, and Beyond

Moore's scaling now advances via packaging, not transistors per die.

Chiplets: split a large die into smaller dies on an interposer (e.g., AMD EPYC, Intel Ponte Vecchio). Yield improves, heterogeneous nodes (logic + HBM + I/O) mix, but inter-chiplet BW ($< 100\,\text{GB/s/mm}$) and coherence across dies cost. UCIe aims to standardise die-to-die links.

3-D stacking: HBM stacks DRAM dies vertically with TSVs, giving 1–3 TB/s with short wires. Logic-on-logic stacking promises similar for compute, at thermal cost (heat trapped between dies).

Near/p-in memory: compute near DRAM (Samsung Aquabolt, UPMEM) or in SRAM arrays to dodge the memory wall; gains are 2–10× for memory-bound kernels when movement dominates. Analog crossbars even do matmul in-memory via Ohm's law, at the cost of precision and noise.

Carbon and cost add new metrics: embodied carbon of manufacturing vs. operational carbon of running. A chiplet that improves yield reduces embodied carbon; a more efficient DSA that halves energy wins operational carbon. Future architects optimise performance/CO_2, not just FLOPS.

Beyond: silicon photonics for rack-scale BW, wafer-scale engines (Cerebras), and quantum accelerators for specific domains. The common thread is still the roofline—move data less, compute more efficiently–and the Iron Law—all optimisations must reduce $IC \times CPI \times T_{\text{clk}}$ or energy × time.

Listing 8.2: EDP trade-off: race-to-sleep vs. DVFS low-power.

```python
# Compare two operating points for a fixed workload (10 G cycles)
cycles = 10e9
def metrics(freq_GHz, volt, power_W):
    time = cycles / (freq_GHz*1e9)
    energy = power_W * time
    edp = energy * time
    return time, energy, edp

A = metrics(3.0, 1.0, 100)  # high freq, high power
B = metrics(1.5, 0.7, 30)   # DVFS: half freq, lower V -> cubic saving
print(f"A time {A[0]:.2f}s E {A[1]:.1f}J EDP {A[2]:.1f}")
print(f"B time {B[0]:.2f}s E {B[1]:.1f}J EDP {B[2]:.1f}")
# B uses less energy but may have worse EDP if latency matters
```

8.6 Exercises

Exercise 8.1. Iron Law: program has IC = 10^9, CPI = 1.5, f = 3 GHz. Compute time. A new compiler halves IC but raises CPI to 2.2; a new pipeline doubles f to 6 GHz but raises CPI to 2.0. Which wins? Why is reporting only f or IC misleading?

Exercise 8.2. Roofline: peak 400 GFLOP/s, BW 25 GB/s. Compute AI_{ridge}. Kernel X AI = 1.5, Y AI = 12: classify each and estimate attainable GFLOP/s. How much must tiling improve X's AI to make it compute-bound? Sketch on the roofline.

Exercise 8.3. Amdahl vs. Gustafson: 90% parallelisable workload on 16 cores. Compute Amdahl speedup.

If weak scaling doubles problem size and parallel fraction grows to 95%, what scaled speedup does Gustafson predict? When does each law apply?

Exercise 8.4. Energy/EDP: CPU A 100 W, 0.5 s vs. B 50 W, 0.8 s. Compute energy and EDP for each. Which wins on energy, which on EDP, and which would you choose for a latency-sensitive server vs. a throughput batch farm?

Exercise 8.5. Dark silicon and chiplets: why can't we power all transistors at max f anymore? How do chiplets, 3-D HBM, and near-memory computing each address the memory wall? Give one drawback of each (BW, thermals, programmability) and relate to the roofline.